

SOLID principi u dijagramu klasa sistema Smart Hair Salon

1. SRP – Single Responsibility Principle

Svaka klasa u sistemu ima jasno određenu odgovornost. Klasa Korisnik sadrži osnovne podatke i funkcionalnosti vezane za korisnika sistema, dok klase Klijent, Administrator i Zaposlenik proširuju njenu funkcionalnost prema ulozi korisnika.

Klasa Rezervacija je odgovorna samo za podatke i ponašanje vezano za rezervaciju termina, dok klasa Termin upravlja informacijama o vremenu i dostupnosti termina. Klasa Proizvod opisuje proizvod, Narudžba obrađuje narudžbu, a Korpa služi za privremeno dodavanje proizvoda prije naručivanja.

Na ovaj način nijedna klasa nema previše različitih zadataka, čime je ispoštovan princip jedne odgovornosti.

2. OCP – Open/Closed Principle

Sistem je otvoren za proširenje, ali zatvoren za izmjene. Osnovna klasa Korisnik može se proširiti novim tipovima korisnika bez mijenjanja postojećeg koda. Trenutno iz nje nasljeđuju klase Klijent, Administrator i Zaposlenik.

Ako se u budućnosti doda nova uloga, na primjer Menadzer, moguće je kreirati novu klasu koja nasljeđuje Korisnik, bez promjene postojećih klasa.

Također, moguće je dodavati nove usluge, proizvode, salone i termine bez promjene osnovne strukture sistema.

3. LSP – Liskov Substitution Principle

Klase Klijent, Administrator i Zaposlenik mogu se koristiti svuda gdje se očekuje objekat tipa Korisnik, jer sve one predstavljaju posebne vrste korisnika.

Svaka izvedena klasa zadržava osnovno ponašanje klase Korisnik, ali dodaje svoje specifične metode. Na primjer, Klijent može kreirati rezervaciju, Administrator može upravljati proizvodima i uslugama, a Zaposlenik može pregledati raspored i obraditi rezervaciju.

Zbog toga nasljeđivanje ne narušava logiku sistema.

4. ISP – Interface Segregation Principle

U sistemu klase imaju samo metode koje su im zaista potrebne. Klijent nema metode za upravljanje proizvodima, jer je to odgovornost administratora. Administrator nema metode za kreiranje rezervacije kao klijent, a zaposlenik nema metode za kreiranje narudžbi.

Na ovaj način klase nisu opterećene nepotrebnim funkcionalnostima. Svaka klasa sadrži samo metode koje odgovaraju njenoj ulozi u sistemu.

5. DIP – Dependency Inversion Principle

Klase u sistemu ne zavise direktno od konkretnih lista atributa, već su veze između klasa prikazane asocijacijama u dijagramu. Naprimjer, Narudzba nije definisana preko atributa lista_proizvoda, nego preko veze sa klasom Proizvod.

Isto važi i za klasu Korpa, koja je povezana sa proizvodima preko asocijacije. Time se smanjuje zavisnost između klasa i sistem postaje lakši za proširenje i održavanje.

U složenijoj implementaciji, metode kao što su dodajProizvod(), kreirajRezervaciju() i posalji() mogu koristiti posebne servise ili interfejse, čime bi se dodatno smanjila zavisnost od konkretnih klasa.